

DEEP LEARNING SUMMER SCHOOL 2026

Foundations of Sequence Modelling

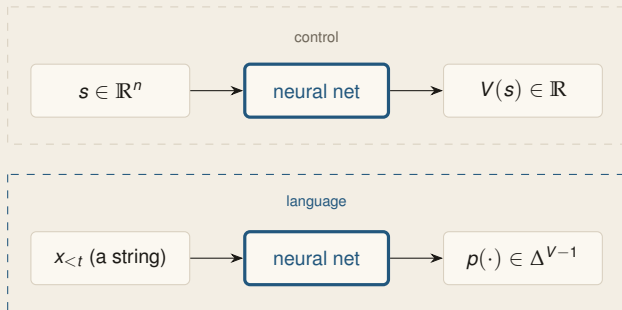
From value functions to token distributions

One question, all session: how does an observed prefix become a distribution over what comes next?

Markus Leippold • University of Zurich & Swiss Finance Institute • Session I of III

Where you are

You already do this — on a different state space



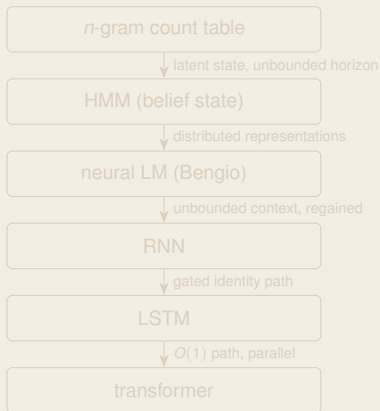
You already approximate value and policy functions on continuous state spaces.

Here the state is a **string** and the output is a distribution over $V \approx 5 \times 10^4$ discrete symbols.

Same approximation problem. What changes is the geometry of the output — and that change is what this deck is about.

The route

Six rungs, one defect each



Every model today estimates the *same* object and reads it out through the *same* head. Only the **context map** changes.

The comparison rule

A rung earns its place only by removing one named defect of the rung below. If it removes nothing, it is decoration.

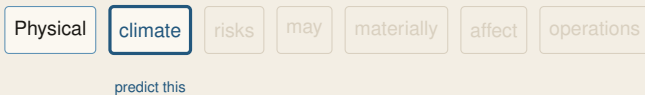
This figure returns at every transition, so you always know where we are.

I The shared problem

*Fix the object, derive its loss once, and separate scoring a token
from measuring the world.*

The estimand

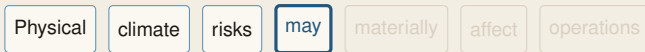
A sentence, one token at a time



A model of a sequence is a distribution over it. Each step conditions on everything to its left — and nothing to its right.

The estimand

A sentence, one token at a time

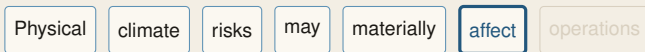


predict this

The window advances. The *target* is always one token; the **context** is everything already observed.

The estimand

A sentence, one token at a time



predict this

Nothing about this depends on the architecture. Every model in this deck is one way of turning the blue boxes into a distribution over the next one.

The estimand

The chain rule makes the factorization exact

Physical

climate

risks

may

materially

affect

operations

predict this

$$p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t \mid x_{<t})$$

Exact, not an approximation — and the reason one next-token problem is the *whole* problem. Remember this product; it comes back as a picture at the causal mask.

The head

One local error signal, for every model in this deck

Collect in $\theta \in \mathbb{R}^P$ every trainable parameter — the **context map** $x_{<t} \mapsto h_t$ and the **unembedding** $U \in \mathbb{R}^{V \times d}$, $z_t = Uh_t$; the same θ acts at every position. The readout's *form* never changes, and the loss is Shannon information (Shannon, 1948):

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}, \quad \ell = -\log p_y = -z_y + \log \sum_j e^{z_j}.$$

Differentiate the two terms separately:

$$\frac{\partial \ell}{\partial z_i} = -\mathbf{1}[i = y] + p_i.$$

$$\nabla_z \ell = p - e_y$$

Predicted minus observed

A residual at the logits: each unobserved token loses the mass it was given; the observed one gains the rest. Identical for every model today.

θ reaches ℓ only through z_t , so the parameter gradient factors:

$$\nabla_{\theta} \ell_t = J_t^{\top} (p_t - e_{y_t}), \quad J_t = \frac{\partial z_t}{\partial \theta} \in \mathbb{R}^{V \times P}.$$

The right factor is universal; every model today is a different J_t — where all the trouble in this deck lives. (J_t is never formed: backprop computes $J_t^{\top} v$ directly.)

The head

The same readout can serve three different targets

Target	What the probability is over
next token	symbols in a vocabulary
continuation	complete strings
state of the world	events a decision depends on

The head gives you the first. Finance wants the third.
Nothing in the softmax converts one into the other.

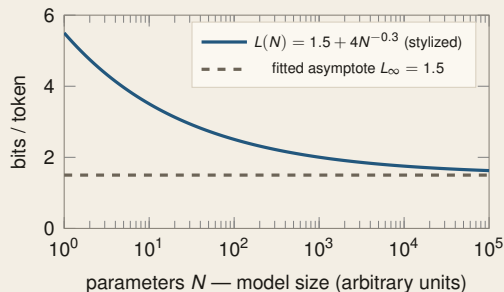
The distinction to carry all week

A next-token probability is a probability over *strings*, not over *states of the world*.

Getting from one to the other needs a map from language to events, and that map has to be estimated and validated like any other measurement.

The score

Cross-entropy separates the floor from the misfit



Training buys the gap, never the floor. The curve is an envelope over models trained at each size, so L_∞ is *fitted* — the entropy-rate frame separates it from H_∞ .

Write $p_\theta(\cdot \mid x_{<t}) = \text{softmax}(z_t)$ for the model's forecast, and let $p^\star$ be the **true** conditional law at that prefix.

A single realized $\ell = -\log p_\theta(y)$ is a draw. Average it under $p^\star$ — the law that generates y — and it becomes a property of the *forecast*:

$$\begin{aligned} R(\theta \mid x_{<t}) &= \mathbb{E}_{y \sim p^\star} [-\log p_\theta(y)] \\ &= \underbrace{H(p^\star)}_{\text{irreducible}} + \underbrace{\text{KL}(p^\star \parallel p_\theta)}_{\geq 0 \text{ (Gibbs)}}. \end{aligned}$$

Only the second term depends on θ : minimizing cross-entropy *is* minimizing KL. Averaging over prefixes as well leaves the split intact, term by term.

The score

What one token costs — the unit, worked

The forecast at one prefix, and the token that actually followed:

token	p_θ
may	0.40
could	0.25 ← observed
will	0.15
might	0.10
(49 996 others)	0.10

$$\ell = -\ln 0.25 = 1.39 \text{ nats} = 2 \text{ bits,}$$

$$\text{PP} = 4.$$

Three ways to say one number

nats — $-\ln p$ — what the gradient uses

bits — $-\log_2 p = -\ln p / \ln 2$

perplexity base-free, and over a corpus

$$\text{PP} = \exp \left(\frac{1}{T} \sum_t -\ln p_\theta(x_t | x_{<t}) \right) = 2^{\bar{H}}$$

Read PP as an **effective branching factor**: the number of equally likely options that would cost the same.

	bits	PP
uniform over $V = 5 \times 10^4$	15.6	50 000
fair coin	1.0	2
certain and right	0	1

The score

Why the KL term is never negative

Proposition 1 (Gibbs' inequality)

For distributions p, q on a finite set,

$\text{KL}(p\|q) = \sum_i p_i \log \frac{p_i}{q_i} \geq 0$, with equality if and only if $p = q$.

Proof. Apply Jensen to the *concave* log:

$$\begin{aligned} -\text{KL}(p\|q) &= \sum_i p_i \log \frac{q_i}{p_i} \leq \log \sum_i p_i \frac{q_i}{p_i} \\ &= \log \sum_i q_i = \log 1 = 0. \end{aligned}$$

Strict concavity forces equality only when q_i/p_i is constant, i.e. $p = q$. (Sum over $\{i : p_i > 0\}$; if some such $q_i = 0$, $\text{KL} = +\infty$ and the bound is trivial.) $\square$

What this buys

The decomposition

$\mathbb{E}_{p^*}[-\log p_\theta] = H(p^*) + \text{KL}(p^*\|p_\theta)$ now has a **floor** you cannot argue with.

A model that reports its own uncertainty honestly cannot be beaten by one that does not — and the gap between them is exactly a KL divergence.

The score

Proper, local — and therefore essentially forced

A scoring rule $S(p, y)$ is **proper** if truth-telling is optimal: $\mathbb{E}_{y \sim p^*} S(p, y)$ is minimized at $p = p^*$. For $S = -\log p_y$ that is Gibbs again,

$$\mathbb{E}_{p^*}[-\log p_y] = H(p^*) + \text{KL}(p^* \| p) \geq H(p^*),$$

and *strictly* so: the minimizer is unique.

It is also **local** — it reads the forecast only through the probability given to what actually happened. The Brier score $\sum_i (p_i - \mathbf{1}[i=y])^2$ is proper but *not* local: it also charges you for how you spread the remaining mass.

Why V is the hypothesis, not colour

On two outcomes Brier *is* local — it collapses to $2(1 - p_y)^2$, so the theorem has nothing to separate. At $V \approx 5 \times 10^4$ it bites: your score depends on the ranking of 49 999 words never in play.

Among strictly proper *local* rules — smooth, ≥ 3 outcomes — the log score is unique up to affine rescaling (Bernardo, 1979). Not an inherited convention: fix incentive-compatibility and locality on a large discrete space and there is nothing left to choose.

Accuracy is neither. The adaptation session cashes that.

The head

Softmax is the maximum-entropy readout, not a convention

Among all distributions on $\{1, \dots, V\}$, which is **least committal** — highest entropy — given what we know?

$$\begin{array}{ccc} \max_p & - \sum_i p_i \log p_i & \text{subject to} \\ \underbrace{\sum_i p_i = 1}_{\text{it is a distribution}}, & \underbrace{\sum_i p_i z_i = \mu}_{\text{the scores mean something}} & . \end{array}$$

This gives $-\log p_i - 1 - \alpha + \lambda z_i = 0 \implies p_i \propto e^{\lambda z_i}$. Multiplier λ is pinned by the *moment* constraint — it solves $\mathbb{E}_{p_\lambda}[z] = \mu$; normalization is α 's job. Softmax is this family at the **scale convention** $\lambda = 1$.

Same argument, other names: the Gibbs distribution, the exponential family, and — at $V = 2$ — the logit.

Two readings of one formula

λ is the **inverse temperature**, and $\lambda = 1$ is where the model was *fitted*. Sampling at $T = 1/\lambda$ re-reads the *same* logits at a different scale — the same exponential family, not a different functional form.

So temperature changes the distribution you *sample* from. It does not change the fitted model, and it does not change z — which is why the bigram lab's held-out perplexity will not move when you turn the dial.

The head

The head is convex; all the hardness is in the encoder

Proposition 2

$\ell(z) = -z_y + \log \sum_j e^{z_j}$ is convex in z , with Hessian

$$H = \text{diag}(p) - pp^\top, \quad v^\top H v = \text{Var}_p(v) \geq 0.$$

Differentiate $\nabla_z \ell = p - e_y$ once more; the quadratic form is a **variance**, hence non-negative — the same calculation, under a different measure, returns as $\text{Var}(q \cdot k) = d_k$ at the attention scaling.

So the objective is convex in z , and the readout is convex in U . Neither is where training gets hard.

Convex, but not strictly

$H\mathbf{1} = 0$. Two familiar consequences:

- the logits are identified only **up to a shift** — the additive gauge, absorbed by the normalization multiplier α a slide ago, not by the scale λ ;
- on **separable** data the optimum recedes to infinity, which is why a penalty is not optional.

None of this is the hard part. Vanishing gradients, initialization, adaptive steps — all of it comes from the **context map** $x_{<t} \mapsto z_t$, never from the loss.

The floor

Every stationary source has an entropy rate

For a stationary process the two limits exist and agree:

$$H_{\infty} = \lim_n \frac{1}{n} H(X_{1:n}) = \lim_n H(X_n \mid X_{<n}).$$

Conditioning cannot increase entropy, so $H(X_n \mid X_{<n})$ is non-increasing in n and bounded below — longer context never hurts *in expectation*.

H_{∞} is a property of the **language**, not of any model.

What this rules out

No architecture, no amount of data, and no amount of compute drives held-out log loss below H_{∞} . Reports of “superhuman” perplexity are reports about a tokenizer and a corpus.

It also disciplines a scaling-law fit $L(N) = L_{\infty} + aN^{-\alpha}$: the fitted asymptote is conditional on the family, tokenizer and training recipe, so $L_{\infty} \geq H_{\infty}$ — it *may approach* the floor under ideal consistency assumptions, and never crosses it. The reference deck measures the approach.

II Counting, and the wall

Counting works, and then it hits a wall that is a law rather than bad luck.

Counting

The bigram is a table you can read

$c(w_{t-1}, w_t)$	climate	credit	risks	may
physical	412	3	8	0
material	27	119	44	0
climate	0	0	2180	6
risks	1	0	0	731

Counts from a filings corpus (stylized). Estimate by normalizing each row.

The first context map is a lookup:

$$\hat{p}(w_t | w_{t-1}) = \frac{c(w_{t-1}, w_t)}{c(w_{t-1})}.$$

- exact, interpretable, and estimated in one pass;
- every cell is a separate parameter — nothing learned about *climate* helps *environmental*;
- and most of the table is zero.

Counting

“Count and normalize” is the maximum-likelihood estimate

Fix a context v . With $n_w = c(v, w)$, $N = c(v) > 0$, and the **empirical** row $\hat{p}_w = n_w / N$, the multinomial log-likelihood — coefficient dropped, no π in it — is

$$\ell(\pi) = \sum_w n_w \log \pi_w = -N[H(\hat{p}) + \text{KL}(\hat{p} \parallel \pi)].$$

$H(\hat{p})$ does not involve π , so maximizing ℓ is minimizing $\text{KL}(\hat{p} \parallel \pi)$, and Gibbs — already earned — puts the unique minimizer at $\hat{\pi} = \hat{p} = c(v, \cdot) / c(v)$. No Lagrangian, no cases: the zeros arrive free, since $\hat{p}_w = 0$ wherever $n_w = 0$.

The score frame with one distribution swapped: there, KL to the **truth**; here, KL to the **sample** — and every neural rung is this same minimization, with π confined to the softmax image.

The zeros are the point

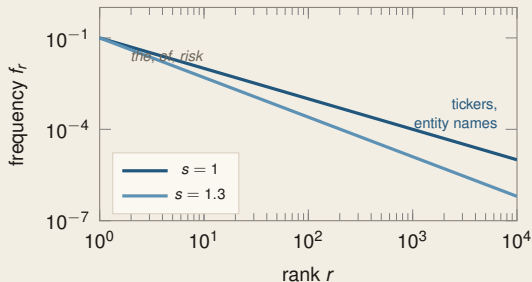
The table is mostly zeros, and a softmax cannot reproduce them: $\hat{\pi}_w = 0$ is only *approached*, as $z_w \rightarrow -\infty$.

So the count table's MLE is a point no neural model can represent — an **accidental regularizer** against infinite held-out loss.

Rows never interact: V separate problems — the no-sharing defect. And $c(v) = 0$ leaves no row at all, a different failure from a zero *inside* a row.

The wall

Zipf makes sparsity structural, not bad luck



Slope = $-s$: the r -th word is about $1/r$ as frequent as the first.

A name's frequency is capped by how often *one* entity is discussed; *the* is shared by every sentence. So in filings the rarest tokens are exactly the ones a decision is about.

Rank the types by frequency, over a vocabulary truncated at rank R :

$$f_r \propto r^{-s}, \quad s \approx 1.$$

- at $s = 1$ the partial sums $\sum_{r \leq R} r^{-1} \sim \log R$ **diverge** — the tail never thins out, and the law does not normalize on an unbounded vocabulary;
- so every new document keeps introducing types you have never seen.

It is not that a *particular* zero survives more data. The **sparsity** does.

The wall

Almost every context you meet is unseen

With $V = 5 \times 10^4$ types there are $V^2 = 2.5 \times 10^9$ bigram cells. A corpus of 10^9 tokens holds at most 10^9 *distinct* bigrams, so coverage is under 40% — and far under, since common pairs recur: *of the* fills one cell a million times over. Trigrams cost another factor of V .

	parameters
bigram table	2.5×10^9
GPT-2 small	1.2×10^8

Twenty times the parameters, and it cannot finish a sentence.

Two responses, one of which is a dead end

Smooth the table — Good–Turing, Kneser–Ney (Good, 1953; Kneser and Ney, 1995) — and you redistribute mass without ever learning that *climate* and *environmental* behave alike.

Or replace the table with a function that **shares parameters** across contexts.

Kneser–Ney is a genuinely strong baseline (Chen and Goodman, 1999) — competitive with early neural LMs, still used where data is scarce. Neural models overtook it; the interesting part is **why**, not by how much. The axis is not capacity but **structure**: what parameters may share.

Live

Counting, smoothing and temperature — with the counts visible

Predict first

Hold the corpus and seed fixed and *raise the temperature*. Does held-out perplexity rise, fall, or stay put?

Then move one control, and name the invariant.

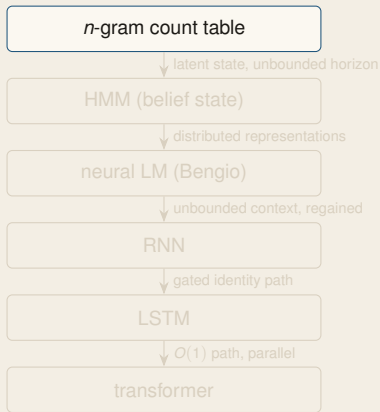
- order n : fluency improves, coverage collapses;
- smoothing: where the unseen mass comes from;
- temperature: changes the *samples*, not the fitted table.

The counts are on screen and auditable — this is the last model today of which that is true.

▷ [live lab: shannon-bigram.html](#)

The route

Defect 1 — the context is a fixed window of words



What is broken

Context is the last $n - 1$ words; nothing before them exists. Widening is not an option — the table grows by a factor of V each time, and coverage collapses faster than fluency improves.

What removes it

Summarize the whole prefix into a **latent state**: a small object, carried forward and updated word by word, that can hold information from arbitrarily far back.

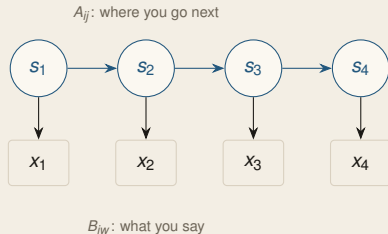
The table's other defect — every cell its own parameter — survives this rung. Next.

III Latent state

*Compress the past into a latent state — an unbounded horizon,
bought with a bottleneck and still built from tables.*

Latent state

Put a hidden variable behind the words



K hidden states; given the state, the word is independent of everything else.

Two tables instead of one:

$$A_{ij} = p(s_t=j \mid s_{t-1}=i), \quad B_{jw} = p(x_t=w \mid s_t=j).$$

What this buys

The n -gram's context is the last $n - 1$ words. Here it is a distribution over K hidden states — a learned summary that can carry information from **arbitrarily far back**.

Think of the states as unobserved modes: *boilerplate*, *risk narrative*, *quantitative*. Nobody labels them; they are whatever best explains the text.

Latent state

The belief state is the context map

We never see s_t , so carry a **belief** about it — a posterior, updated one word at a time:

$$b_t(i) = p(s_t = i \mid x_{1:t}) \in \Delta^{K-1}.$$

Prediction is one push through the two tables:

$$p(x_{t+1}=w \mid x_{1:t}) = \sum_j \left(\sum_i b_t(i) A_{ij} \right) B_{jw}$$

The definition sums K^T paths; the **forward recursion** evaluates the same quantity at $O(TK^2)$, because b_t is **sufficient**. $K=8$, $T=200$: $8^{200} \approx 10^{181}$ terms against $200 \times 8^2 = 12,800$ multiply-adds for the whole pass.

The deck's thesis in miniature

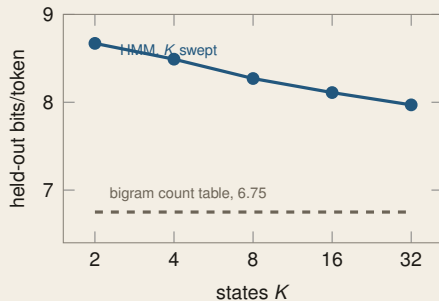
b_t is the context map. Every model from here on is a different answer to “what do I carry forward?”

Every rung answers **what is sufficient to carry forward**. The RNN carries d free numbers — sufficient only if the cell learned to make them so. The transformer refuses to summarize, and pays $O(n^2)$.

Not literally a softmax — a mixture of the K emission rows, already normalized — but it fills the same slot: write $z_w = \log p(w \mid x_{1:t})$ and $p - e_y$ returns.

Latent state

The rung, measured



40 real 10-Ks, 3.5M tokens. Gains per doubling of K : 1.14, 1.16, 1.11, 1.10 — not monotone, but trending toward 1.

Read it honestly

The HMM *loses to the bigram* on perplexity at every K . This rung buys *horizon*, not sharpness — and the flattening sweep is the bottleneck becoming visible.

Fitted with $K = 8$, the states organise themselves into boilerplate, statement language, sector vocabulary and numerics — *no labels anywhere*.

Both claims of the defect frame, measured before it is asserted.

colabs/latent_state_rung.ipynb \$5-\$7 — executed, committed with outputs
▷ live lab: *latent_state_rung.ipynb*

Latent state

You have seen this recursion before

The update splits in two, and it is Bayes' rule once per period:

$$p(s_t=j \mid x_{<t}) = \sum_i b_{t-1}(i) A_{ij}$$

predict: push through the dynamics

$$b_t(j) \propto p(s_t=j \mid x_{<t}) B_{jx_t}$$

update: reweight by what you just saw

Same recursion, different emission

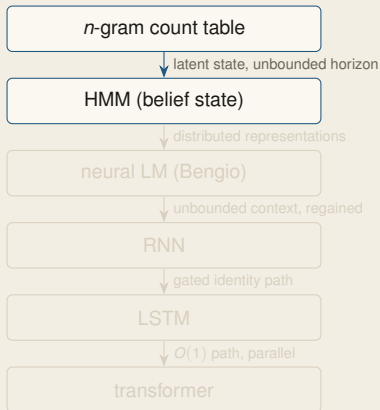
Continuous state, Gaussian noise: the [Kalman filter](#). Discrete state, Gaussian returns: [Hamilton's Markov-switching model](#) (Hamilton, 1989).

This is the regime filter you already use in finance, with a word in place of a return — typically with B replaced by a normal density.

Run live in the companion notebook: a planted two-regime return process comes back with σ recovered to two decimals, 98% state accuracy, and calm/turbulent durations of 75 and 27 days.

The route

Defect 2 — the horizon is fixed, the tables are not



What is broken

A and B are still **lookup tables**. Nothing learned about *climate* reaches *environmental* — two columns of B with nothing between them. Defect 1 removed the window; the sharing defect survived it.

What removes it

Give each word a **vector**, and let one shared function map context vectors to logits. Similar words then borrow each other's evidence.

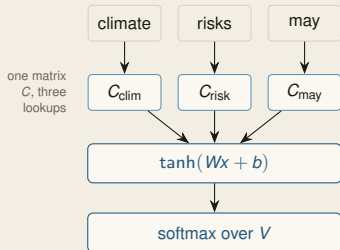
The cost: that function has a fixed window again. The horizon is not recovered until the RNN.

IV Representations

Words become vectors, and prediction becomes the signal that trains them.

Bengio

Replace the table with a function



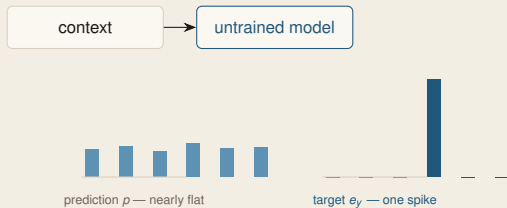
Each word gets a vector $C(w) \in \mathbb{R}^m$; one hidden layer maps the concatenated context $x_{\text{ctx}} = [C(w_{t-n+1}), \dots, C(w_{t-1})]$ to logits (Bengio et al., 2003):

$$z = U \tanh(Wx_{\text{ctx}} + b).$$

- parameters grow with m and V , not with V^n ;
- the **same** head and **same** $p - e_y$ sit on top;
- gradients reach C too, so the vectors are learned, not given.

Training

“thou shalt ____ make a machine in the likeness of a human mind.” - Frank Herbert, Dune



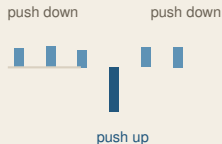
Feed the context to an untrained model and ask for the neighbouring word.

The output is a probability over the whole vocabulary. Because nothing has been learned, it is nearly **flat** — and almost certainly wrong.

The correct answer is one spike.

Training

thou shalt ____ make a machine — the error vector



$p - e_y$: every wrong token loses mass, the observed token gains it.

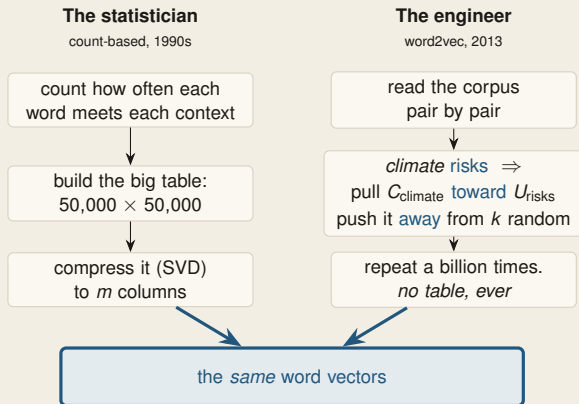
Subtract the two vectors and you get exactly the head's gradient $p - e_y$ from Section I:

$$\nabla_z \ell = p - e_y$$

- one coordinate is negative — the token that actually occurred;
- every other coordinate is positive and small;
- the update flows back into W and into C , so the vectors move too.

Embeddings

Two very different recipes, one answer



word2vec keeps the two matrices we already have: the embedding C , one row C_w per word, and the unembedding U , one row U_c per context (Mikolov et al., 2013).

Levy and Goldberg (2014) asked what that really optimizes. At the optimum,

$$U_c^\top C_w = \text{PMI}(w, c) - \log k$$

PMI is *pointwise mutual information*, $\log \frac{p(w, c)}{p(w)p(c)}$: zero when w and c are independent, positive when they meet more often than chance. The statistician's cell, exactly.

What word2vec bought

Not new information — the same counts. *Speed*.

Delete every *unfunded* → *plans* from a 14M-token filings corpus (44 events, ± 12 -token windows), refit **both** models on the same vocabulary, and ask each for the deleted pair on held-out contexts:

	MLP p	rank of <i>plans</i>	count p
full corpus	0.071	4	0.033
ablated	0.009	19 of 20,000	exactly 0

The table has **no other route** to the pair. The MLP retains 12.8% of the probability — **183×** the best smoothed table's floor.

How the evidence traveled

The learned neighbours of *unfunded*: *underfunded*, *contributory*, *qualified*, *inactive*, *funded*, *nonqualified* — the pension-plan adjective class, built from co-occurrence alone. Evidence about any of them reaches all of them.

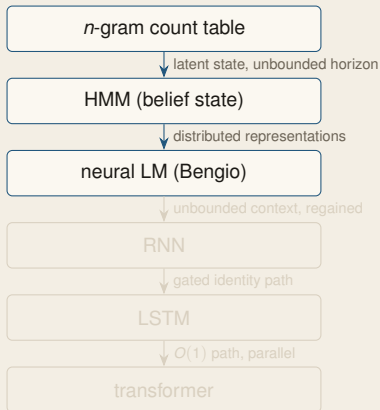
The rung's claim, measurable: distributed representations move probability onto events the table must score zero.

colabs/bengio_rung_gpu.ipynb §6b-§7 — executed on GPU, committed with outputs

▷ live lab: [bengio_rung_gpu.ipynb](#)

The route

Defect 3 — one vector per type, and a fixed window



What is broken — two things

The window is back. Context is the last $n - 1$ words; the horizon we bought at Defect 1, traded away for sharing.

One vector per type. The same $C(w)$ serves “*material weakness*” and “*raw material*” — it lands between the senses and fits neither.

What removes the window

A recurrent **state**: the latent-state idea again, now with learned vectors instead of a table.

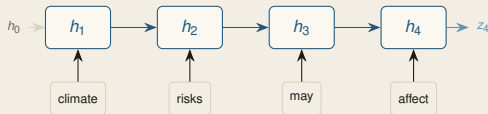
The ambiguity needs a *context-dependent* representation. It survives until self-attention.

V Recurrence

Recurrence gives an unbounded horizon — and a product of Jacobians that will not behave.

Recurrence

Fold the whole prefix into a state



One shared cell applied repeatedly. h_t is the belief state again — but a free point in $\mathbb{R}^d$, not a distribution on the simplex.

$$h_t = \tanh(a_t), \quad a_t = W_h h_{t-1} + W_x x_t + b,$$

$$z_t = U h_t.$$

- the formal horizon is now **unbounded** — h_t depends on every earlier token;
- the parameter count does not grow with sequence length;
- h_t is the HMM's b_t with the constraint removed and the update **learned, not derived**: d free numbers instead of $K - 1$ probabilities — more capacity, no longer readable, and sufficient only *if training made it so*;
- but still a *fixed-dimensional* summary of an ever-longer past (Elman, 1990).

Recurrence

Backpropagation through time is a product of Jacobians

The loss at step t reaches an input k steps earlier only through the chain

$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{j=t-k+1}^t \frac{\partial h_j}{\partial h_{j-1}}$$

Differentiate $h_j = \tanh(a_j)$: the outer $\tanh$ is coordinate-wise, so its derivative is the *diagonal* of slopes; a_j is linear in h_{j-1} , so its derivative is W_h . Outer first:

$$\frac{\partial h_j}{\partial h_{j-1}} = \text{diag}(\tanh'(a_j)) W_h.$$

W_h is the same every time — one shared cell — but the diagonal is **not**: it depends on the trajectory.

Why this object, and not a sum

A product of k matrices behaves nothing like a sum of k terms. Its size is a product of gains, and products do only two things.

Keep the trajectory dependence in view. It is what turns the coming bound into an *inequality* rather than an equality.

Recurrence

Why long products destabilize — one inequality

Write $\gamma_j = \|\text{diag}(\tanh'(a_j))\|$ for the *per-step* gain and $\sigma = \sigma_{\max}(W_h)$ for the largest singular value — the **operator norm**. Submultiplicativity, factor by factor:

$$\left\| \frac{\partial h_j}{\partial h_{j-1}} \right\| \leq \gamma_j \sigma \implies \left\| \frac{\partial h_t}{\partial h_{t-k}} \right\| \leq \sigma^k \prod_j \gamma_j.$$

Bound every γ_j by $\gamma = \sup_j \gamma_j \leq 1$:

$$\left\| \frac{\partial h_t}{\partial h_{t-k}} \right\| \leq (\gamma \sigma)^k$$

- $\gamma \sigma < 1$ is **sufficient** for vanishing — geometric, so long-range credit is not slow, it is *absent*;
- $\gamma \sigma > 1$ is **inconclusive**: a bound that permits growth does not produce it;
- $\tanh$ cannot help. $\sup |\tanh'| = 1$, so $\gamma = 1$ is the *best* case; saturation shrinks γ_j and makes vanishing worse (Bengio et al., 1994; Pascanu et al., 2013).

Operator norm, not spectral radius: $\sigma_{\max} \geq \rho(W_h)$ always; normality is sufficient for equality, not necessary.

Live

Move $\sigma_{\max}$, watch the gradient

Predict first — commit out loud

At $\sigma_{\max}(W_h) = 1.05$ the bound $(\gamma\sigma)^k$ *permits* growth. At lag 50, does the measured gradient vanish, explode, or stay put?

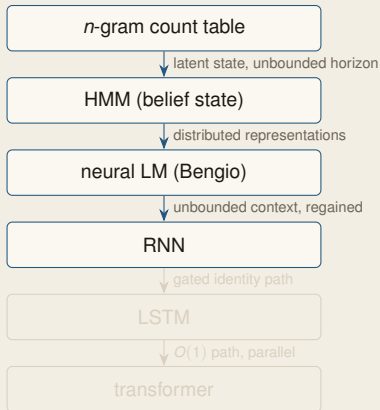
A 10-K sentence runs 40 tokens. A filing runs tens of thousands. The dependency you care about sits at the lag where the gradient is not.

- measured: **twelve orders of magnitude down**. An upper bound above one is permission, not prediction — products of random matrices fall far below it;
- explosion needs $\sigma_{\max} \gtrsim 2.3$ here, so vanishing is not a knife edge. It is nearly everywhere;
- clipping flattens explosion and leaves vanishing untouched — so the fix cannot be an optimizer.

▷ *live lab: [rnn-machine.html](#)*

The route

Defect 4 — the gradient dies before the dependency does



What is broken

The horizon is unbounded on paper and unreachable in practice. Credit travels back through a **product** of Jacobians, and the product decays geometrically — measured, twelve orders of magnitude by lag 50.

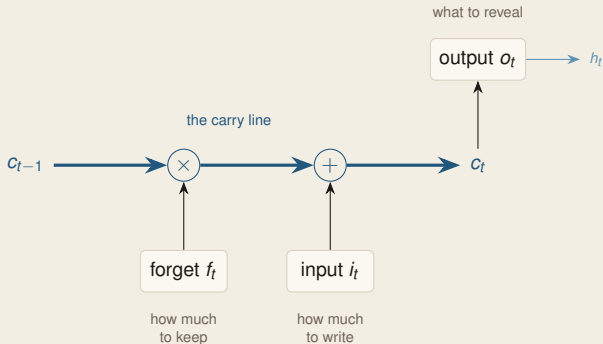
What removes it

An **additive** route the state can travel unchanged: a gated identity path, so at least one coordinate has Jacobian near 1 however long the sequence.

Not an optimizer fix. Clipping bounds explosion and leaves vanishing exactly where it was.

LSTM

An identity path for the cell state



$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad h_t = o_t \odot \tanh(c_t).$$

Two lines, two jobs

c_t is **memory**, written additively. h_t is a **view** of it, gated by o_t . Separating storage from readout is the innovation — not the number of gates (Hochreiter and Schmidhuber, 1997).

With $f_t \approx 1$ and $i_t \approx 0$ the cell copies itself: $\partial c_t / \partial c_{t-1} \approx 1$.

The forget gate is not in the 1997 paper — it arrives three years later (Gers et al., 2000). Not a new rung: same recurrence, same fixed state. Gates repair the *Jacobian* so the rung becomes trainable.

The recurrence on c is additive, so the *direct* path has Jacobian $\text{diag}(f_t)$ instead of a repeated dense matrix. Hold the gate near one and that path stops decaying.

LSTM

Why an additive path changes the Jacobian

Differentiate the cell update along the *direct* route:

$$\frac{\partial c_t}{\partial c_{t-1}} = \text{diag}(f_t) + R_t,$$

where R_t collects the indirect paths $c_{t-1} \rightarrow h_{t-1} \rightarrow$ gates. The direct route is a product of diagonals, hence diagonal, and its norm is *coordinatewise*:

$$\left\| \prod_j \text{diag}(f_j) \right\| = \max_r \prod_j |f_{j,r}|$$

Diagonals near one, not dense matrices of norm $\gamma\sigma$ — and *one* coordinate held near one suffices, which is exactly what a learned gate can arrange.

State the claim precisely

The full Jacobian is *not* diagonal: R_t is real. What gates buy is the **existence** of a route whose Jacobian is near the identity, so gradient can reach far back even when every other route has died.

A highway does not uncongest the side streets. It means there is a way through.

Remember the device, not the arithmetic: the transformer's residual stream is this again, across *depth* instead of time.

LSTM

Clipping fixes exactly one of the two failures

Gradient clipping rescales without rotating — acting on the **parameter** gradient assembled at the end of backprop, not on the per-step Jacobians inside the product:

$$g \leftarrow g \cdot \min \left(1, \frac{\tau}{\|g\|} \right).$$

- **Explosion** — bounded, so one bad batch cannot destroy the parameters, and the direction survives, so the step is still descent;
- **Vanishing** — untouched. Clipping only ever *shrinks* a gradient. It cannot manufacture signal that arrived as 10^{-12} .

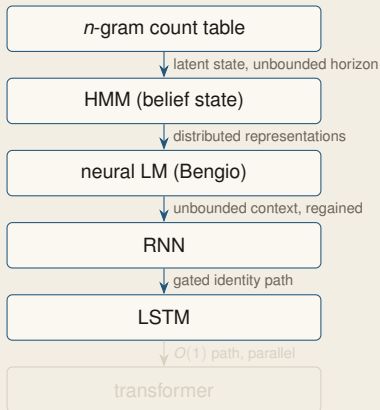
The asymmetry is the point

One failure is an *optimizer* problem with an optimizer fix. The other is an **architecture** problem, and no optimizer will ever reach it.

You saw this in the lab: clipping flattened the explosive branch and left the vanishing one exactly where it was.

The route

Defect 5 — a fixed state must decide what to forget



What is broken

Gates fixed the **gradient**, not the **bandwidth**. Everything the model may later need still fits through one vector h_t — written *before* it knows what will be asked.

What removes it

Stop compressing. Keep every position, and let each step **retrieve** what it needs: a lookup whose keys are content.

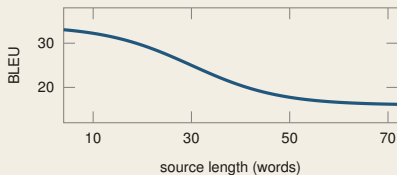
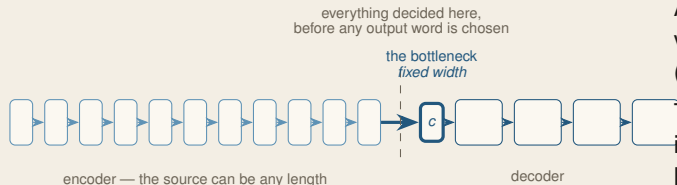
Sequential training goes too — and that, not capacity, is what made the next rung scale.

VI Attention

Replace compression with retrieval: attention makes context query-dependent and the path length one.

seq2seq

Everything through one vector



Stylized — the shape a fixed-capacity channel predicts, and the one measured (Cho et al., 2014). Short sentences are fine; the same model degrades as the source grows.

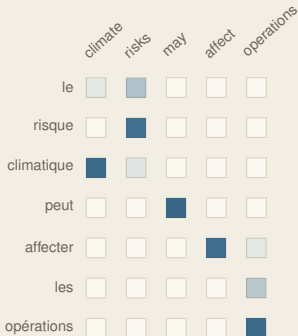
An encoder reads the source into one vector c ; a decoder generates from it (Sutskever et al., 2014).

The dimension of c is fixed. The source is not. So quality degrades with source length — a **structural** failure, not a training one: the same model translates short sentences fine.

Not a new rung. Every other frame on the ladder removes a named defect. This one removes nothing — it is the *previous* defect, made visible in the smallest case where you can point at it. The rung is the next frame.

Attention

A learned, content-based lookup



English **source** across, French **output** down — attention was introduced for translation. Learned from sentence pairs alone, never from alignments (stylized).

Replace the single c with a *query-dependent* summary (Bahdanau et al., 2015):

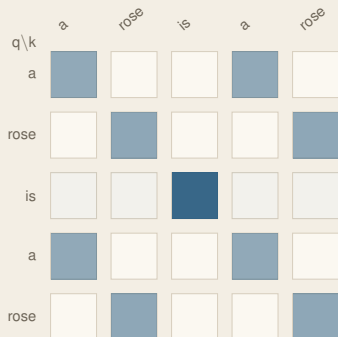
$$\alpha_{tj} = \text{softmax}_j (\text{score}(h_{t-1}, \bar{h}_j)),$$
$$c_t = \sum_j \alpha_{tj} \bar{h}_j.$$

And c_t enters the decoder as one more input:
 $h_t = f(h_{t-1}, y_{t-1}, c_t)$.

- $\bar{h}_j$: the encoder's state at source position j . $\alpha_{t.}$: a **distribution over those positions**, so c_t is a content-chosen average — nothing discarded in advance;
- score is a small learned net, fit by the same cross-entropy — two rungs on it becomes $q \cdot k / \sqrt{d_k}$.

Attention

Who supplies Q , K and V ?



"a rose is a rose" — each token attends to its twins.

Three projections of the *same* stream, then a row-wise softmax of similarities:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

$$\text{Attn}(X) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

- **self**-attention: Q , K , V all from one sequence;
- **cross**-attention: Q from the decoder, K , V from the encoder.

Attention

Why divide by $\sqrt{d_k}$

Let the entries of q and k be independent, mean zero, unit variance. Then

$$q \cdot k = \sum_{i=1}^{d_k} q_i k_i, \quad \text{Var}(q \cdot k) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = d_k.$$

So the logits have standard deviation $\sqrt{d_k}$ and grow with the head dimension.

Dividing by $\sqrt{d_k}$ restores unit scale (Vaswani et al., 2017).

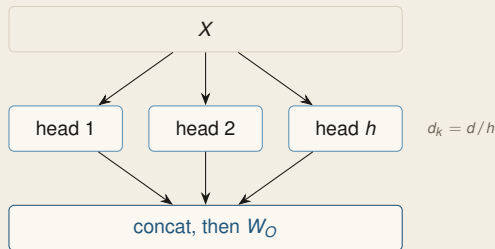
Why it matters, not just why it is there

Softmax on large logits saturates: one weight goes to 1, the rest to 0, and the gradient through the softmax goes to **zero**.

Without the scaling, deeper models with wider heads simply stop training — a temperature bug dressed as an architecture constant.

Attention

Several relations at once



Split d into h subspaces, attend in each, concatenate, project.

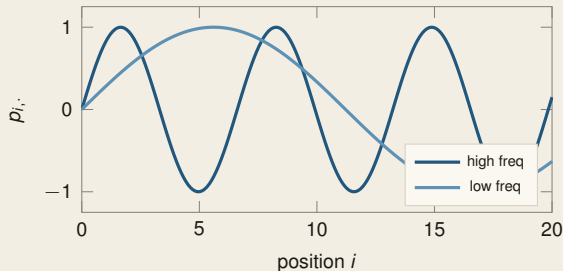
- total cost is unchanged — the heads *divide* the width, they do not add to it;
- one head can track agreement while another tracks a distant qualifier.

Say this out loud

Heads are parallel learned subspaces. They are **not** guaranteed to carry interpretable roles, and published head-labels are post hoc.

Position

Without it, attention sees a set



Geometric frequencies $\omega_\ell = 10000^{-2\ell/d}$ — one per coordinate pair; RoPE rotates the same ℓ instead of adding it.

Permute the input rows and every attention output permutes with them. So with no position signal, “risks may not materialize” and “risks may materialize not” — the **same four tokens**, reordered — give each token the same representation in both.

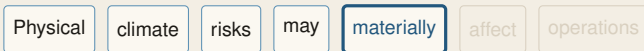
Sinusoids make similarity depend on the **offset** alone,

$$p_i \cdot p_j = \sum_{\ell} \cos((i - j)\omega_{\ell}),$$

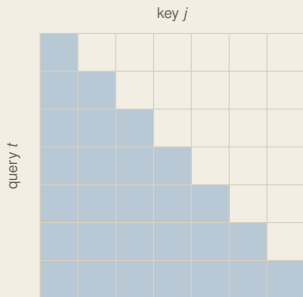
and RoPE achieves the same by rotating q and k (Su et al., 2021).

Position

The mask is the chain rule



predict this



Shaded: allowed, $j \leq t$. Blank: masked.

The triangle is the product we wrote at the start,

$$p(x_{1:T}) = \prod_t p(x_t \mid x_{<t}),$$

enforced in code — with one explicit shift: position t sees $x_{\leq t}$ ($j \leq t$), so its output is trained against the *next* token x_{t+1} . Setting the upper triangle to $-\infty$ before the softmax is not a trick — it is the [information set](#).

In finance this is the same discipline as an as-of date. Break it and you have look-ahead — which is exactly how it will break in the agentic session, through the retrieval index rather than the labels.

Position

Permutation equivariance, proved

Proposition 3

For any permutation matrix P , unmasked self-attention satisfies $\text{Attn}(PX) = P \text{Attn}(X)$.

Proof. The projections are row-wise, so $Q \mapsto PQ$, $K \mapsto PK$, $V \mapsto PV$. Then

$$(PQ)(PK)^\top = PQK^\top P^\top,$$

and since softmax acts row-wise, conjugation by P passes through it: $\text{softmax}(PSP^\top) = P \text{softmax}(S)P^\top$. Hence $\text{softmax}(PSP^\top)PV = P \text{softmax}(S)V$. $\square$

Say it exactly

Equivariance, not invariance: outputs permute *with* inputs, so the layer is not a set function. What it forbids is order-sensitivity — a token's representation depends on its own identity and the **multiset** of the rest, never their arrangement.

Two ways out, both used here: add a position signal to X , or break the symmetry structurally — the **causal mask** is tied to indices, so masked attention is not equivariant either.

Attention

It is kernel-weighted regression — for this audience, the fastest route in

Put $\kappa(q, k) = \exp(q^\top k / \sqrt{d_k})$. Then a single query's output is

$$\text{Attn}(q) = \frac{\sum_j \kappa(q, k_j) v_j}{\sum_j \kappa(q, k_j)}$$

which is exactly the **Nadaraya–Watson** estimator (Nadaraya, 1964; Watson, 1964) of $\mathbb{E}[v \mid q]$ with an exponential inner-product kernel.

What is different from 1964

Classical kernel regression *chooses* the kernel and the bandwidth. Attention **learns** the features (W_Q, W_K) and fixes the bandwidth at $\sqrt{d_k}$.

The curse of dimensionality that kills kernel smoothers in $\mathbb{R}^d$ is dodged the same way it always is — by learning a low-dimensional representation in which the smoothing is done.

The payoff

In-context learning is implicit Bayes — and you have seen the recursion

Suppose pretraining data is a mixture over latent tasks θ , $p(x) = \int p(x | \theta) \pi(\theta) d\theta$. A model reproducing that *joint* — not merely fitting the marginal — has, by Bayes, exactly one conditional:

$$p(x_{\text{next}} | \text{prompt}) = \int p(x_{\text{next}} | \text{prompt}, \theta) \underbrace{p(\theta | \text{prompt})}_{\text{posterior over tasks}} d\theta$$

If θ is also **identified** by the prompt, the posterior concentrates and the predictive approaches $p(\cdot | \text{prompt}, \theta^*)$ — the few-shot behaviour of Brown et al. (2020), read as Bayes (Xie et al., 2022).

The callback

This is the **belief state** again. In the HMM, each observation updated a posterior over hidden states. Here, each prompt token updates a posterior over latent tasks — and the weights never move.

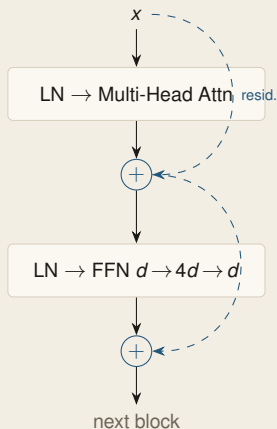
Two hypotheses do real work here: specification and identification. A low marginal loss implies neither.

VII The transformer, assembled

Assemble the block, price the trade you accepted, and hand the unfixed defect to the next deck.

The block

Residual, normalization, feed-forward



Pre-norm form: normalize, apply the sub-layer, *add back* (Xiong et al., 2020).

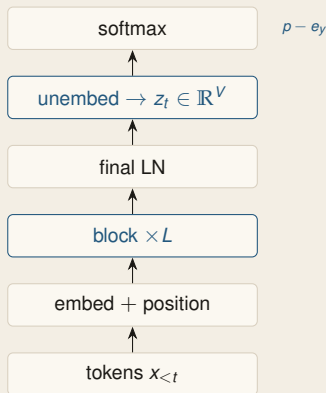
$$x \leftarrow x + \text{MultiHead}(\text{LN}(x)),$$

$$x \leftarrow x + \text{FFN}(\text{LN}(x)).$$

- attention mixes **across** positions; the FFN computes **within** one;
- the residual makes each sub-layer an *edit* to a shared stream — the LSTM's carry line, now across depth rather than time.

The block

Decoder-only, end to end



One stream. Prompt and continuation are concatenated, so source and target stop being separate objects — the cross-attention case drops out, and every block here is self-attention under the mask.

Every rung of the ladder ends here: a context map producing z_t , then the *same* softmax and the *same* residual.

- training is parallel across t — all positions scored in one pass, which is why this architecture scaled and recurrence did not;
- generation is still sequential, one token at a time;
- weight tying often shares the embed and unembed matrices.

The trade

Cost $O(n^2)$, path $O(1)$

	RNN / LSTM	transformer
compute per layer	$O(nd^2)$	$O(n^2d + nd^2)$
sequential steps	$O(n)$	$O(1)$
max path length	$O(n)$	$O(1)$

The transformer is *more* expensive in arithmetic and *cheaper* in the only currency that mattered: sequential depth on a parallel device.

Where the quadratic bites

Counting a layer properly — $8nd^2$ of projections, $16nd^2$ of feed-forward, $4n^2d$ of attention — the quadratic term overtakes the rest at $n \gtrsim 6d$. At $d = 4096$ that is $n \approx 24,600$: short filings are dominated by the position-wise terms, a full 10-K is not.

That crossover is why long-context engineering fills a chapter of the reference deck rather than a footnote.

Objectives

Two pretraining objectives on one architecture

Causal (GPT). Predict every next token:

$$\mathcal{L}_{\text{CLM}} = - \sum_t \log p(x_t \mid x_{<t}).$$

Masked (BERT). Corrupt a subset M and reconstruct it from *both* sides (Devlin et al., 2019):

$$\mathcal{L}_{\text{MLM}} = - \sum_{t \in M} \log p(x_t \mid x_{\setminus M}).$$

Same blocks, same head, same $p - e_y$. Only the mask pattern and the summation set differ.

The signal-per-token ledger

CLM supplies a gradient at **every** position. MLM masks about 15%, so roughly 1/7 of the positions carry loss — and it pays for that with genuinely bidirectional context.

A trade, not a ranking: bidirectional evidence versus supervision density.

▷ *live lab: Inside_BERT notebook*

Objectives

Why decoders won the generative race — and where encoders still win

The causal objective *is* the chain-rule factorization the deck opened with. So a trained decoder is immediately a sampler: generate x_t , append, repeat.

A masked model gives you a family of conditionals with **no guarantee** that any joint distribution has them all as its conditionals. Compatibility is not automatic, and trained MLMs are measurably incompatible — so there is no exact way to sample a document from one.

But the ledger has two columns

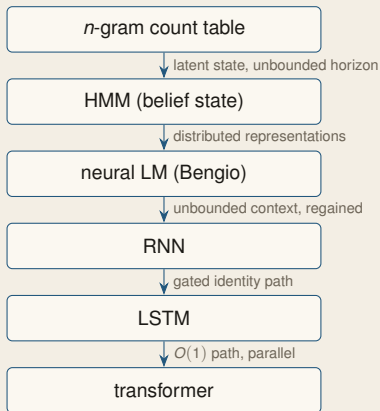
When the task is **classification with labels** and the text is short, a bidirectional encoder is still competitive at a fraction of the parameters — and it is cheap enough to fine-tune on a laptop.

Which is exactly why the adaptation session starts from FinBERT rather than from a frontier model. “Use the biggest decoder” is a recommendation, not a theorem.

The encoder, opened up — the `Inside_BERT` notebook in `colabs/` rebuilds the forward pass tensor by tensor (verified against the reference), watches the heads with BertViz, and gets 83% sentiment accuracy from frozen features.

The route

Defect 6 — one pass, fixed context, no evidence



What is still broken

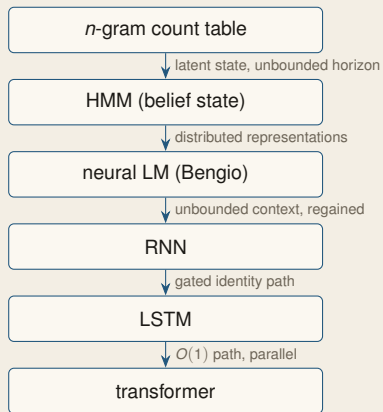
Retrieval removed the bottleneck *inside* the context window. It did not give the model more **knowledge**, a way to consult a **document**, or any notion of *when* a fact was true.

Everything left is an allocation problem: how much model, how much data, how much evidence, how much inference.

That is the next three decks — three allocation problems.

The route

Six rungs, each removing one named defect



- **count table:** exact counts; a window of symbols, sparse by law.
- **HMM:** the past compressed into a belief; A and B still tables.
- **neural LM:** vectors and one shared function — sharing bought, horizon traded away.
- **RNN:** the horizon returns; gradients vanish in time.
- **LSTM:** an additive carry line rescues the gradient; one bottleneck remains.
- **transformer:** content-addressable retrieval; $O(1)$ path, parallel.

Through all six: **one head, one loss, one gradient $p - e_y$** — and from Bengio on, also the *training signal*.

Carry forward

What the head gives you is over strings

Return to Section I's three-targets frame. After every rung of the ladder the readout is unchanged, and so is what it ranges over:

$$p(\text{next symbol} \mid \text{prefix})$$

Not $p(\text{the firm is exposed} \mid \text{filing})$. The second requires a map from language to events — estimated, and validated, separately.

The habit to build

Whenever a number comes out of a language model, ask: *which event is this a probability about?*

We ask it again of a benchmark score (the reference deck), of an AI-generated label (Session II), and of an authorized action (Session III).

Backup appendix

*Optional derivations and detail, preserved for questions and
handout use — not part of the taught path.*

Backup

RoPE: rotate, and relative position falls out in two lines

Split q, k into $d/2$ coordinate pairs and rotate pair ℓ at position m by angle $m\omega_\ell$; write R_m for the block-diagonal rotation.

Proposition 4

$(R_m q)^\top (R_n k) = q^\top R_{n-m} k$: the score depends on positions *only through the offset* $n - m$.

Proof. Rotations are orthogonal with $R_m^\top = R_{-m}$, and $R_a R_b = R_{a+b}$ blockwise. Hence $(R_m q)^\top (R_n k) = q^\top R_m^\top R_n k = q^\top R_{-m} R_n k = q^\top R_{n-m} k$. $\square$

Absolute in, relative out

Nothing is added to the residual stream; the rotation is applied *inside* the attention score. Translation invariance is therefore exact, not learned (Su et al., 2021).

It is defined at any length — which is not the same as working at any length. Extrapolation beyond the trained range degrades, and the reference deck returns to why.

Backup

Universal approximation, and what it does not say

Proposition 5 (informally)

*A one-hidden-layer network is **dense** in $C(K)$, K compact. Cybenko (1989) proves it for continuous sigmoidal activations; Leshno et al. (1993) sharpen it: density holds iff the activation is non-polynomial.*

True, and almost useless alone. It is silent on all three quantities you pay:

- **width** — no bound, possibly exponential in dimension;
- **data** — nothing about estimating it from samples;
- **optimization** — nothing about whether SGD finds it.

The argument it cannot make

“Transformers work because they are universal approximators” explains nothing. So is a shallow net, and so is a big enough lookup table.

Everything interesting is about *which* functions are cheap to represent, cheap to fit, and reachable. That is why the ladder is about resources.

Backup

Three allocation problems

At home	Scale. A fixed compute budget, split between parameters and data. A constrained optimization with a fitted response surface — your take-home reference deck, with its notebook.
Session II	Adaptation. Where domain signal enters, which objective supplies it, how far weights may move.
Session III	Evidence. Point-in-time retrieval, tools, and what you can defensibly publish.

Before the next session, if you want one thing

Open `shannon-bigram.html` and find the order n at which your own writing stops being predictable. It is lower than you think.

Backup

Attention is a differentiable associative memory

Store patterns as the rows of X and query with ξ . The modern Hopfield update

$$\xi \longleftarrow X^\top \text{softmax}(\beta X \xi)$$

is *identical* in form to one attention head with $\beta = 1 / \sqrt{d_k}$.

Proposition 6 (Ramsauer et al.)

*If the stored patterns are well separated, the update retrieves the nearest pattern in **one** step, and the number of patterns storable with high probability grows **exponentially** in the dimension.*

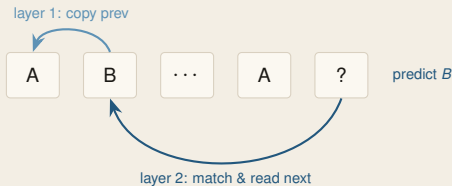
Why this reframing helps

It explains what a head *is* rather than what it does on one example: content-addressed retrieval from a set, with a temperature controlling how sharply one item wins.

Low β : a blended average of many patterns. High β : a hard lookup of one. The temperature you saw in the bigram demo is the same knob, on a different alphabet.

Backup

Induction heads: two layers compose to copy from context



An in-context bigram, assembled from two heads.

- **Layer 1** — a *previous-token* head writes each token's predecessor into its own position;
- **Layer 2** — a head matches the current token *A* against those records and reads off what *followed A* last time.

The bigram, replayed from the prompt

Two attention layers compose into a **bigram-like retrieval** — match the current token, copy what followed it last time, with no weight update. The behaviour of the count table, not the table itself (Elhage et al., 2021).

Backup

What LayerNorm and the feed-forward block each do

LayerNorm standardizes each token's activation vector across features, then rescales. It controls the scale entering each sub-layer and makes the residual stream comparable across depth.

The FFN is the only place where a position computes on itself, and it holds roughly two-thirds of the parameters. Attention routes; the FFN thinks.

Backup

Byte-pair encoding: the vocabulary is learned, not given

Start from characters and repeatedly merge the most frequent adjacent pair. Common words become single tokens; rare ones decompose (Sennrich et al., 2016).

Consequences that matter in finance: tickers and numbers fragment unpredictably, token counts differ across tokenizers, and perplexities are therefore **not** comparable between models.

Backup

A financial token is a modelling choice

The same machinery applies to discretized returns, regime labels, or order-book events. Nothing in the derivation used the fact that x_t was a word.

But the analogy stops at the data-generating process: efficiency flattens the conditional *mean* of returns while leaving the conditional *variance* and the regime predictable. The agentic session returns to this.

References I

- Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *3rd International Conference on Learning Representations (ICLR 2015)*, 2015.
- Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994. doi: 10.1109/72.279181.
- Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. A neural probabilistic language model. *Journal of machine learning research*, 3(Feb):1137–1155, 2003.
- José M Bernardo. Expected information as expected utility. *the Annals of Statistics*, pages 686–690, 1979.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS) 33*, 2020.
- Stanley F. Chen and Joshua Goodman. An empirical study of smoothing techniques for language modeling. *Computer Speech & Language*, 13(4):359–394, 1999.
- Kyunghyun Cho, Bart van Merriënboer, Dzmitry Bahdanau, and Yoshua Bengio. On the properties of neural machine translation: Encoder–decoder approaches. In *Proceedings of SSST-8, Eighth Workshop on Syntax, Semantics and Structure in Statistical Translation*, pages 103–111, 2014. doi: 10.3115/v1/W14-4012.

References II

- George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2:303–314, 1989.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, volume 1, pages 4171–4186, 2019.
- Nelson Elhage, Neel Nanda, Catherine Olsson, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021.
- Jeffrey L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990. doi: 10.1207/s15516709cog1402_1.
- Felix A. Gers, Jürgen Schmidhuber, and Fred Cummins. Learning to forget: Continual prediction with LSTM. *Neural Computation*, 12(10):2451–2471, 2000. doi: 10.1162/089976600300015015.
- Irving J. Good. The population frequencies of species and the estimation of population parameters. *Biometrika*, 40(3–4): 237–264, 1953.
- James D. Hamilton. A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica*, 57(2):357–384, 1989.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.
- Reinhard Kneser and Hermann Ney. Improved backing-off for M-gram language modeling. In *1995 International Conference on Acoustics, Speech, and Signal Processing*, volume 1, pages 181–184. IEEE, 1995.

References III

- Moshe Leshno, Vladimir Ya. Lin, Allan Pinkus, and Shimon Schocken. Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural Networks*, 6(6):861–867, 1993.
- Omer Levy and Yoav Goldberg. Neural word embedding as implicit matrix factorization. In *Advances in Neural Information Processing Systems*, volume 27, 2014.
- Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. In *Advances in neural information processing systems*, pages 3111–3119, 2013.
- Elizbar A. Nadaraya. On estimating regression. *Theory of Probability & Its Applications*, 9(1):141–142, 1964. doi: 10.1137/1109020.
- Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *Proceedings of the 30th International Conference on Machine Learning (ICML 2013)*, pages 1310–1318, 2013.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-1162. URL <https://aclanthology.org/P16-1162>.
- Claude E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948. doi: 10.1002/j.1538-7305.1948.tb01338.x.
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

References IV

- Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems 27 (NeurIPS 2014)*, pages 3104–3112, 2014.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017.
- Geoffrey S. Watson. Smooth regression analysis. *Sankhyā: The Indian Journal of Statistics, Series A*, 26(4):359–372, 1964.
- Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. An explanation of in-context learning as implicit Bayesian inference. *International Conference on Learning Representations*, 2022.
- Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 10524–10533, 2020. URL <https://proceedings.mlr.press/v119/xiong20b.html>.